

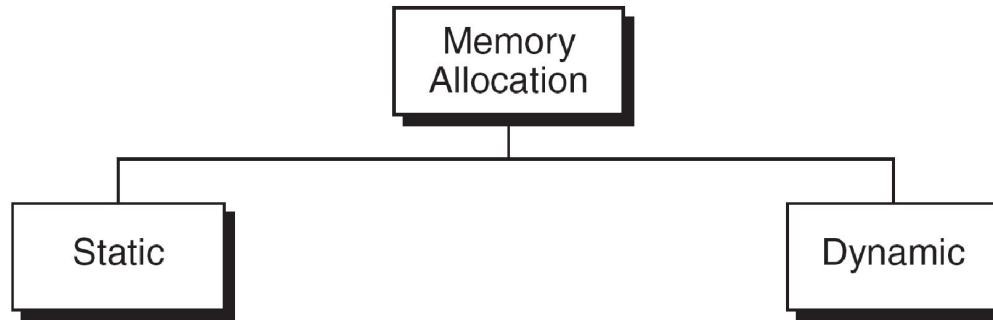


Computer Programming I (CSE2003)

9. Pointer

Memory allocation functions

- C언어를 사용하면서 변수 영역의 메모리를 할당 받는 것으로 다음의 두 가지 방법이 가능하다.

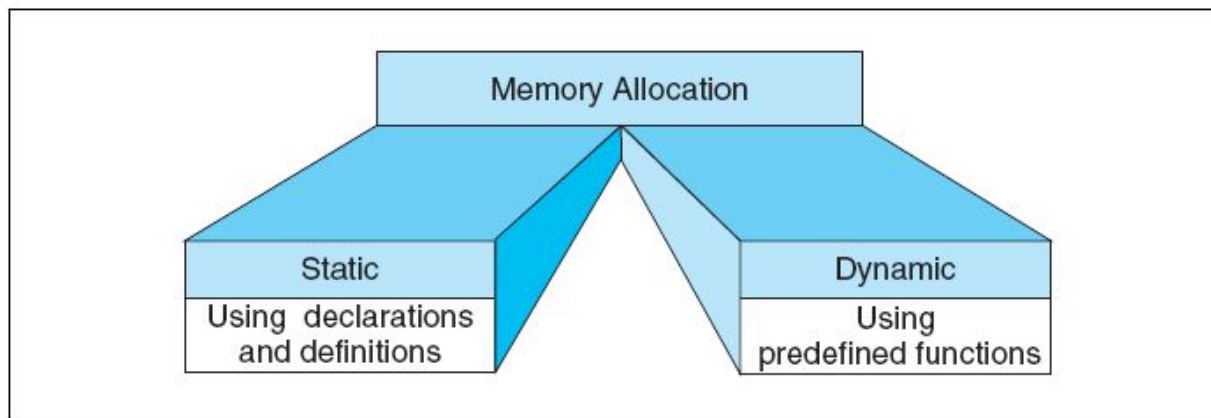


- Static memory allocation (정적 메모리 할당)
 - 처리해야 할 데이터의 크기를 미리 다 아는 경우에는 프로그램에 필요한 메모리의 선언과 정의가 완벽하게 다 명시되어 있어야 한다.
 - 변수, 배열 선언 등
 - `int i, j, a[100][100];`

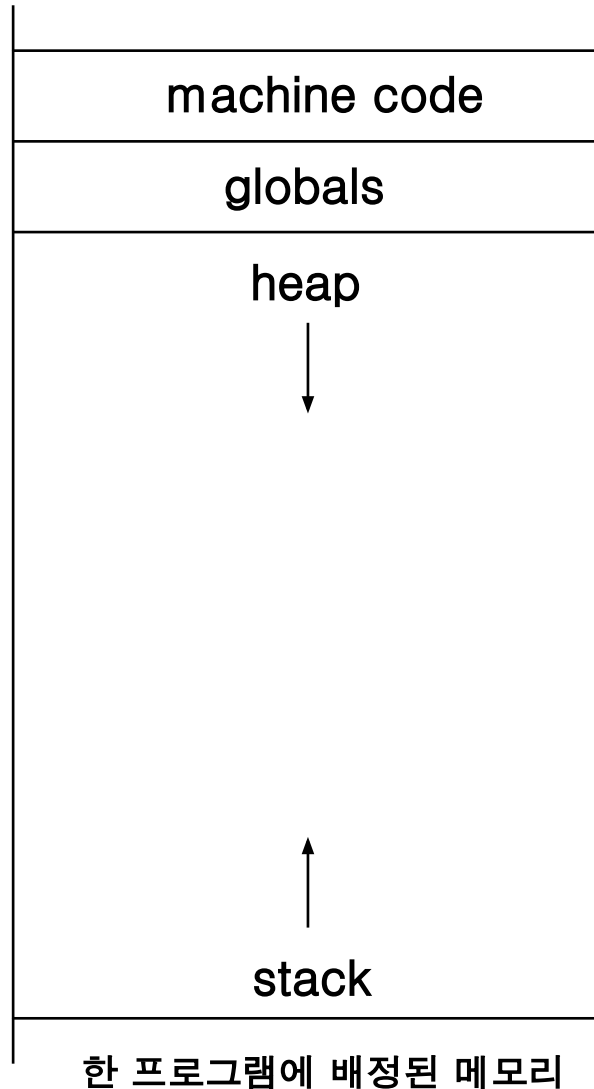
Memory allocation functions

동적 메모리 할당

- 얼마나 큰 데이터를 처리해야 하는지 모르는 경우에는 프로그램의 실행 중에 입력되는 데이터에 맞게 기억공간이 할당되어야 한다. □
메모리의 동적 할당(dynamic memory allocation)
- 메모리의 동적 할당과 해제 기능은 라이브러리로 구현되어 있고, 이를 이용하기 위해서는 반드시 포인터를 이용한다.
 - 동적 할당이란 프로그램 실행 중에 ,즉 run time때 메모리 영역을 할당한다는 뜻이다.



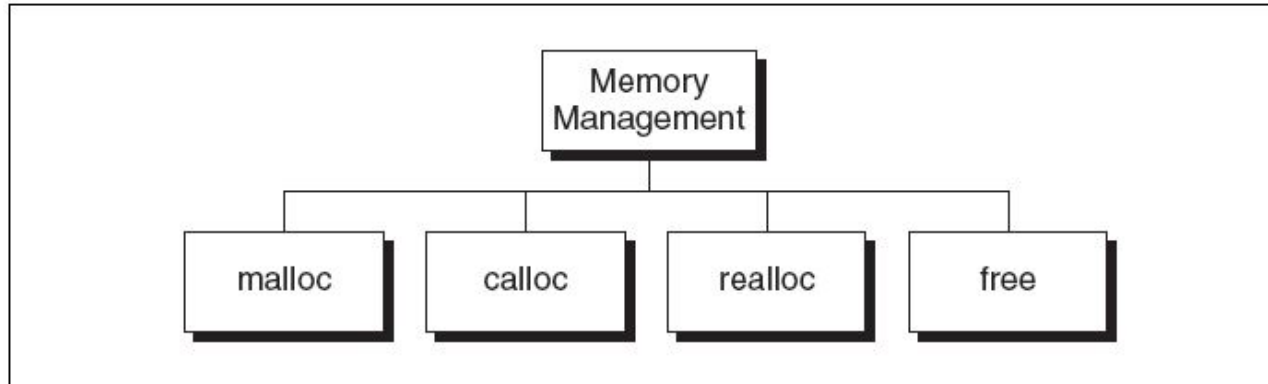
memory 와 프로그램



- ▣ machine code: 컴파일된 프로그램이 올라가는 영역
- ▣ globals: 글로벌 변수들...
- ▣ heap: 동적 할당 메모리 영역
 - malloc()과 같은 동적 메모리 할당 함수가 실행되면 이 영역의 메모리를 가져와서 그것의 주소를 return한다
- ▣ stack은 정적으로 선언된 변수들이 run-time에 배정되는 위치

Memory allocation functions

□ Memory management functions



- 메모리의 동적 할당에 사용되는 메모리 할당 함수들
 - malloc(), calloc(), realloc()은 메모리를 할당하기 위한 함수
 - free()는 더 이상 사용하지 않는 메모리를 리턴하는 함수
- 함수를 사용하기 위해서는 “stdlib.h” 헤더파일을 포함해야 한다.
 - #include<stdlib.h>

malloc()

- malloc() 함수는 동적으로 메모리를 할당하기 위한 함수이다.
 - malloc(n)과 같이 호출하면 n byte의 메모리를 가지고 와서 그것의 시작 주소(pointer)를 return 한다.
 - malloc() 함수는 할당한 메모리의 시작주소를 void 포인터로써 return한다. 할당에 실패한 경우 NULL을 return 한다.
- malloc() 함수의 정의는 다음과 같다

```
void *malloc (size_t size);
```

- 항상 void 포인터를 return하고 파라미터로는 unsigned integer 형태로 받는다.

Dynamic memory allocation

```
// statically obtain an integer
```

```
int x;
```

```
// dynamically obtain an integer
```

```
int *px = malloc(4);
```

```
// dynamically obtain an integer
```

```
int *py = malloc(sizeof(int));
```

Dynamic memory allocation

```
// get an integer from the user
```

```
int x;
```

```
scanf("%u", &x);
```

```
// array of floats on the heap
```

```
float* heap_array = malloc(x * sizeof(float));
```

Dynamic memory allocation

❑ 문제점:

- 동적으로 할당 받은 메모리는 사용한 함수가 종료되어도 자동으로 시스템에 반환되지 않는다!!!
- 동적으로 할당된 메모리를 다시 시스템이 회수하지 못하는 현상을 memory leak라고 하는데, 이렇게 지속적으로 메모리를 회수하지 못할 경우 시스템의 성능이 심각할 정도로 문제가 된다.
- 동적으로 할당된 메모리를 사용할 때, 반드시 해당 메모리를 free() 시켜 주는 것이 중요하다!!!

Dynamic memory 반환

```
char* word = malloc(50 * sizeof(char));
```

```
// word를 가지고 작업 시작
```

```
// word를 가지고 작업 완료
```

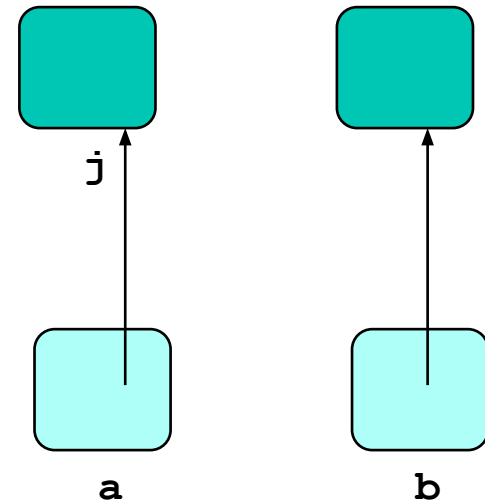
```
free(word)
```

Three golden rules

1. `malloc()`으로 할당 받은 메모리는 반드시 나중에 `free()` 시켜주어야 한다.
2. `malloc()`으로 받은 메모리만 `free()` 시켜주면 된다
3. `free()`는 꼭 한 번만 시행한다.
 - 같은 메모리를 두 번 이상 `free()` 시키면 에러가 남

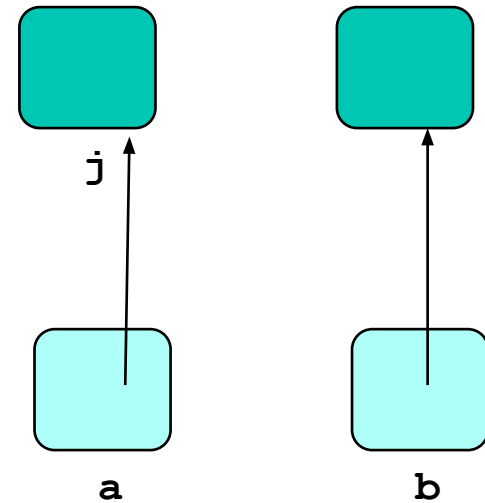
Dynamic memory allocation

```
int j;  
int* a;  
int* b = malloc(sizeof(int))  
a = &j;
```



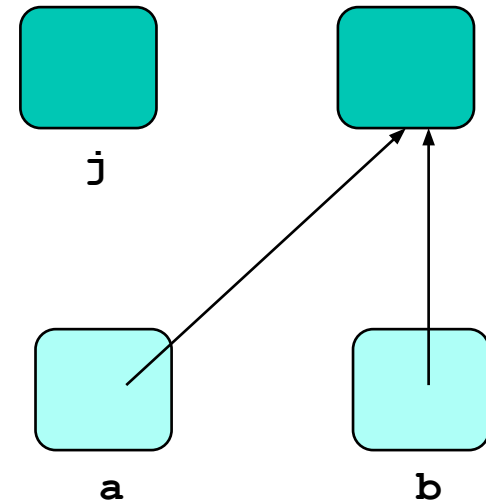
Dynamic memory allocation

```
int j;  
int* a;  
int* b = malloc(sizeof(int))  
a = &j;  
a = b;
```



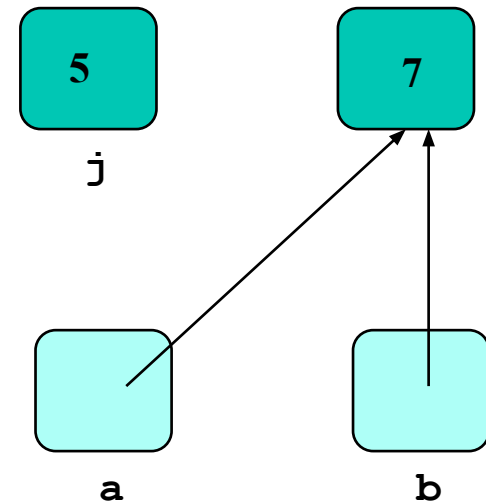
Dynamic memory allocation

```
int j;  
int* a;  
int* b = malloc(sizeof(int))  
a = &j;  
a = b;
```



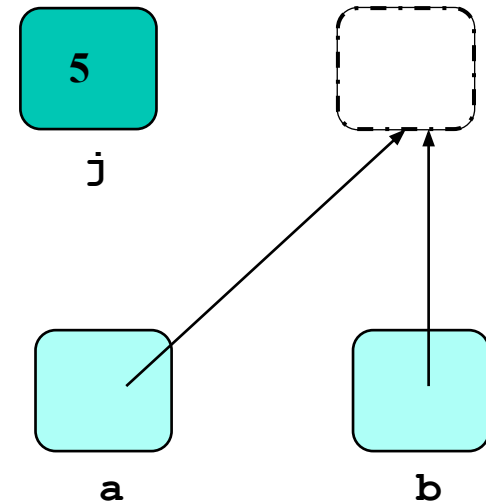
Dynamic memory allocation

```
int j;  
int* a;  
int* b = malloc(sizeof(int))  
a = &j;  
a = b;  
j = 5;  
*b = j + 2;
```



Dynamic memory allocation

```
int j;  
int* a;  
int* b = malloc(sizeof(int))  
a = &j;  
a = b;  
j = 5;  
*b = j + 2;  
free(b);  
*a = 11;
```



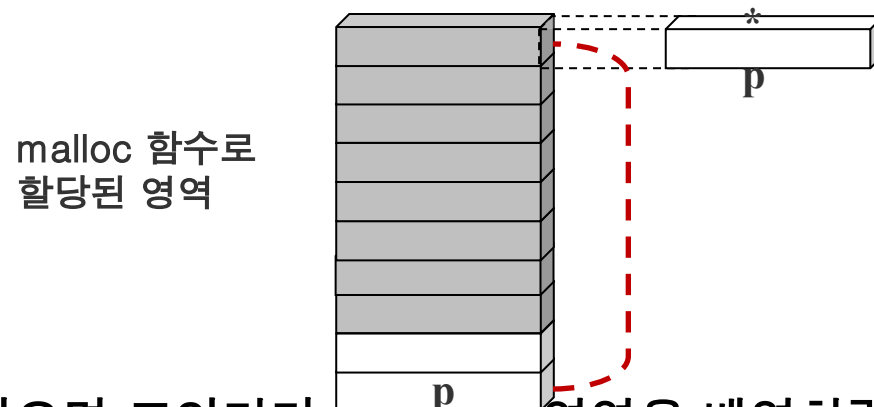
Unpredictable result since it's gone...
May cause segmentation fault, or what...

Dynamic allocation of array

□ Dynamic allocation of array

```
int *p;  
p = (int *)malloc(8 * sizeof(int));
```

- malloc은 8개의 int를 위한 기억 장소를 할당해 그 시작 주소값을 되돌려 준다.



- 포인터 변수가 있으면 포인터가 가리키는 영역을 배열처럼 사용가능
- 포인터 변수 p가 가리키는 원소의 i번째 원소를 나타내는 표현식
 - $*(p+i)$, $p[i]$

malloc()

예제 프로그램 - malloc을 이용한 변수 할당

```
#include <stdio.h>
#include <stdlib.h>
```

```
int main(void)
{
```

```
    int *p;
```

```
    p=(int*)malloc(1*sizeof(int));
```

```
    if(p==NULL) {
```

```
        printf("not allocated\n");
        return 1;
```

```
    }
```

```
    *p=1;
```

```
    printf("%d\n", *p);
```

```
    return 0;
```

```
}
```

```
[root@mclab chap10]# ./chap10-5
```

```
1
```

```
[root@mclab chap10]#
```

int형 1개 크기의 메모리
할당하고 그 주소값을
p에 넘겨준다.

새로 할당된
메모리영역에
접근

p가 새로
할당된 영역을
가리킨다.

malloc 함수로
할당된 영역

malloc()

- 예제 프로그램 - 메모리를 동적으로 할당 받아 배열로 사용하는 프로그램

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main(void)
5 {
6     int *ip;
7     int i, sum = 0;
8
9     ip = (int *)malloc(5*sizeof(int));
10    if(ip==0) {
11        printf("short memory\n");
12        return 1;
13    }
14    printf("Input the age of 5 people : ");
15    for(i=0; i<5; i++) {
16        scanf("%d", ip+i);
17        sum+=ip[i];
18    }
19    printf("Average : %.11f\n", sum/5.0);
20    free(ip);
21    return 0;
22 }
```

20바이트
의
기억공간
할당

```
[root@mclab chap10]# vi chap10-6.c
[root@mclab chap10]# gcc -o chap10-6 chap10-6.c
[root@mclab chap10]# ./chap10-6
Input the age of 5 people : 21 27 24 22 35
Average : 25.8
[root@mclab chap10]#
```

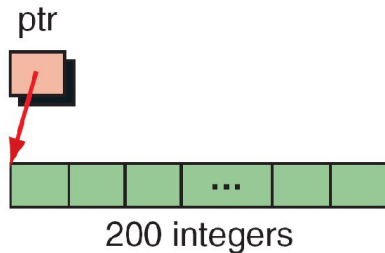
할당 받은
메모리를 배열처럼
사용하여 값을
입력 받는 부분

Contiguous Memory Allocation – calloc()

- ▣ 두 번째 메모리 관리 함수는 calloc() 이다.
- calloc() 함수의 프로토타입은 다음과 같다.

```
void *calloc (size_t element-count,  
              size_t element_size);
```

- 할당 받은 메모리를 사용할 하나의 원소 크기와 전체 원소의 개수를 개별적인 인자로 받는다 .
- calloc() 함수는 메모리를 할당하면서 해당 메모리를 0으로 초기화 시킨다.



```
if (!(ptr = (int*)calloc (200, sizeof(int))))  
    // No memory available  
    exit (100) ;  
  
// Memory available  
...
```

<calloc 함수 호출
예>

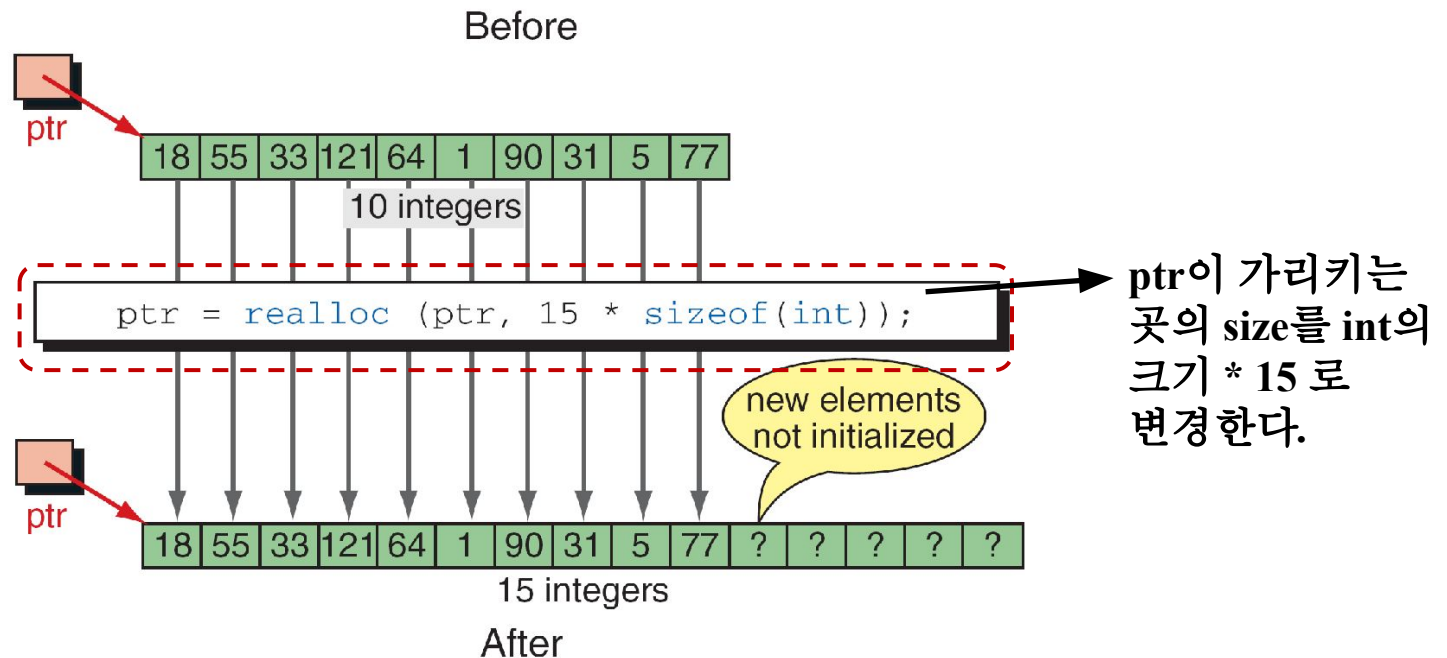
malloc()과
calloc()의 차이점

Reallocation of Memory – realloc ()

- ❑ realloc() 함수는 변수의 메모리를 동적으로 변경하기 위하여 사용한다.
- ❑ 함수의 프로토타입은 다음과 같다.

```
void *realloc (void *ptr, size_t newSize);
```

- ❑ 즉, ptr이 현재 할당하고 있는 메모리의 크기를 newSize로 변경한다.



Releasing Memory – free ()

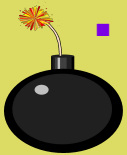
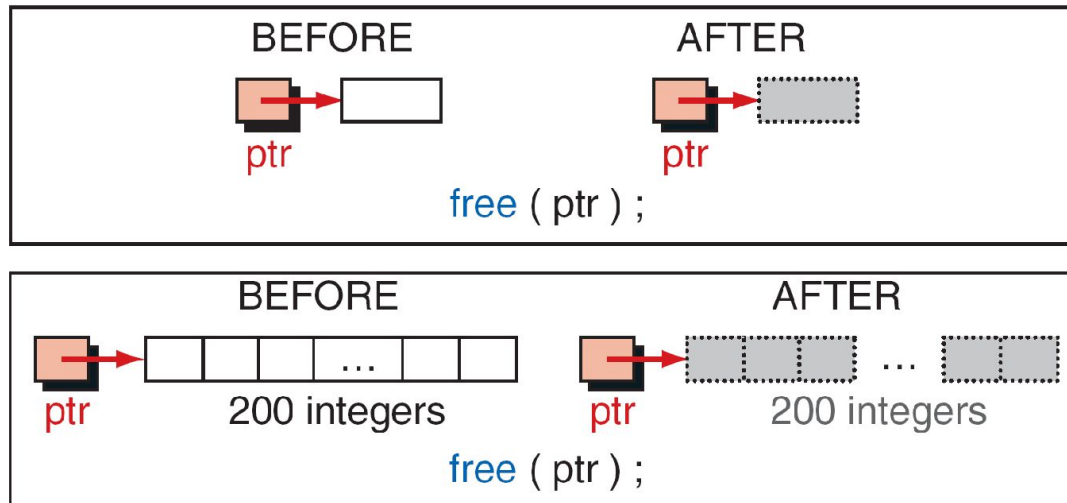
- 동적으로 할당 받은 메모리 영역에 대하여 반환하는 함수
 - free() 함수의 프로토타입은 다음과 같다.

```
void free (void *ptr);
```

- 사용하지 않는 메모리에 대한 계속적인 할당은 메모리 사용의 낭비의 단점이 있기 때문에 동적으로 할당 받은 필요 없는 메모리 영역에 대해서는 free()를 통해서 메모리를 반환하도록 한다.
- free() 는 ptr이 가리키는 기억 장소를 해제한다.
- 단, ptr이 NULL일 때는 아무 일도 하지 않는다.
- ptr은 malloc(), calloc(), realloc()에 의하여 이전에 할당되었던 기억 장소의 포인터이어야 한다.

Releasing Memory – free ()

▣ 다음은 free()의 동작을 보여주는 그림이다.



Caution!

동적으로 할당한 기억 장소 영역은 책임지고 해제하지 않으면 안 된다.
영역을 해제할 때까지는 반드시 포인터를 기억해 두어야 한다.

실습 9-5

실습 9-6